



MULUNGUSHI UNIVERSITY

School of Engineering and Technology | Department of Computer Science

Due Date 5th October 2026: Time 23:59Hrs

ICT371 PostgreSQL Scenario Assignment

Choose any four scenarios

Scenario 1 University Library Book Loans

The university library tracks books and the number of copies available. A student may borrow one or more copies. Each loan is recorded, and returning it restores the available copies.

1. Create a books table with available copies and a book_loans table with student number, quantity and loan status. Add at least three books.
2. Use IF ELSIF ELSE to display whether one book is unavailable, low on copies or sufficiently stocked.
3. Use WHILE to display three overdue reminder numbers. Use numeric FOR to display three library shelf numbers.
4. Create a borrow_book procedure that checks the available copies, reduces stock and records a loan.
5. Call borrow_book for two valid loans and one request exceeding the available copies. Query both tables to show which loans were recorded.
6. Create a return_book procedure that marks a loan returned and restores copies. Call it twice for the same loan; the second call must not restore copies again.
7. Use an explicit cursor to display books with few copies remaining.
8. Try to borrow zero copies and handle the invalid quantity with an EXCEPTION block.
9. Query books and book_loans to show final quantities and loan statuses.

Scenario 2 Computer Laboratory Reservations

A computer laboratory has a fixed number of workstations available for each practical session. Lecturers reserve workstations for their classes. Cancelling a reservation releases those workstations.

1. Create a lab_sessions table with available workstations and a reservations table with lecturer, number of workstations and status. Add at least three sessions.
2. Use IF ELSIF ELSE to report whether a session is full, nearly full or has enough workstations.
3. Use WHILE to print three session preparation reminders. Use numeric FOR to number three workstation checks.
4. Create a reserve_workstations procedure that checks capacity, reduces availability and records a reservation.

5. Call it for two valid reservations and one request that exceeds capacity. Query sessions and reservations after the calls.
6. Create a cancel_reservation procedure that releases workstations. Call it twice for one reservation; the second call must not release them again.
7. Use an explicit cursor to display sessions with few workstations remaining.
8. Request zero workstations and handle the invalid quantity with an EXCEPTION block.
9. Query both tables to show final workstation availability and reservation statuses.

Scenario 3 Student Hostel Room Allocation

A hostel has rooms with a limited number of bed spaces. Students are allocated bed spaces, and checking out releases them. Each allocation records the student and room.

1. Create a hostel_rooms table with available bed spaces and an allocations table with student number, room and status. Add at least three rooms.
2. Use IF ELSIF ELSE to report whether a room is full, has one space left or has several spaces.
3. Use WHILE to show three hostel inspection days. Use numeric FOR to number three room checks.
4. Create an allocate_room procedure that checks for an available space, reduces availability by one and records the student allocation.
5. Call it for two valid student allocations and one allocation to a full room. Query rooms and allocations.
6. Create a check_out procedure that releases a bed space and marks the allocation complete. Call it twice for the same allocation; the second call must not free another space.
7. Use an explicit cursor to display full or nearly full rooms.
8. Attempt an allocation using a blank student number and handle the invalid input with an EXCEPTION block.
9. Query both tables to show final available spaces and allocation statuses.

Scenario 4 Campus Clinic Medicine Dispensing

The campus clinic stores medicines and their available quantities. A clinician dispenses medicine to a student and records the quantity. A dispensing record entered in error can be reversed.

1. Create a medicines table with stock quantities and a dispensing_records table with student number, quantity and status. Add at least three medicines.
2. Use IF ELSIF ELSE to report whether one medicine is out of stock, low on stock or sufficiently stocked.
3. Use WHILE to show three stock review days. Use numeric FOR to number three shelf inspections.
4. Create a dispense_medicine procedure that checks stock, reduces it and records the dispensing action.
5. Call it for two valid quantities and one quantity exceeding stock. Query medicines and dispensing records.
6. Create a reverse_dispensing procedure for an entry made in error. Restore stock and mark it reversed. Call it twice for the same record; stock must be restored only once.
7. Use an explicit cursor to display medicines below a low-stock threshold.
8. Request a negative dispensing quantity and handle the invalid input with an EXCEPTION block.
9. Query both tables to show final stock and dispensing statuses.

Scenario 5 Engineering Workshop Tool Loans

An engineering workshop lends tools to students for practical work. Each type of tool has an available quantity. Returning tools increases that quantity.

1. Create a tools table with available quantities and a tool_loans table with student number, quantity and status. Add at least three tool types.
2. Use IF ELSIF ELSE to report whether a tool is unavailable, low on stock or readily available.
3. Use WHILE to show three workshop safety reminders. Use numeric FOR to number three tool inspections.
4. Create an issue_tool procedure that checks stock, reduces it and records a tool loan.
5. Call it for two valid loans and one request exceeding stock. Query tools and tool_loans to inspect the changes.
6. Create a return_tool procedure that restores stock and marks a loan returned. Call it twice for one loan; the second call must not add stock.
7. Use an explicit cursor to display tools with low availability.
8. Try to issue zero tools and handle the invalid quantity with an EXCEPTION block.
9. Query both tables to show final tool quantities and loan statuses.

Scenario 6 University Event Seat Booking

A university event has a limited number of seats. Students book seats, and a cancelled booking releases them for others.

1. Create an events table with available seats and a bookings table with student number, number of seats and status. Add at least three events.
2. Use IF ELSIF ELSE to report whether an event is full, nearly full or has plenty of seats.
3. Use WHILE to display three booking reminder days. Use numeric FOR to number three entrance checks.
4. Create a book_seats procedure that checks availability, reduces the remaining seats and records a booking.
5. Call it for two valid bookings and one request exceeding the remaining seats. Query events and bookings.
6. Create a cancel_booking procedure that releases the seats and marks the booking cancelled. Call it twice for the same booking; the second call must not release seats again.
7. Use an explicit cursor to display full or nearly full events.
8. Try to book zero seats and handle the invalid quantity with an EXCEPTION block.
9. Query both tables to show final seat availability and booking statuses.

Evidence to submit

Do all this on Postgress Editor pgpql save it as an sql file with extension .sql save each all the four files using the format scenario_number_student_number.sql